I BUILT AN AI DOCUMENT TOOL

AI that shows its work.

Ask a document a question. Every claim in the answer comes with its **page**, the **exact quote**, and a check that the quote is really there.

Q: When must claims be filed, and what is the ER copay?

Within 90 days of the date of service. The ER copay is $150, waived if the member is admitted.

✓ p.2 Verified quote

"Claims must be submitted within 90 days of the date of service."

✓ p.3 Verified quote

"Emergency room visits have a $150 copay that is waived if the member is admitted to the hospital."

Moiz Alizai · DocQA

Swipe →

THE PROBLEM

Confident answers. No sources.

AI chat can summarise a 50-page policy in seconds. But you can't tell which parts are real.

Long documents run the work

Insurance policies, claims manuals, contracts: one clause can take an hour to find.

Answers you can't check

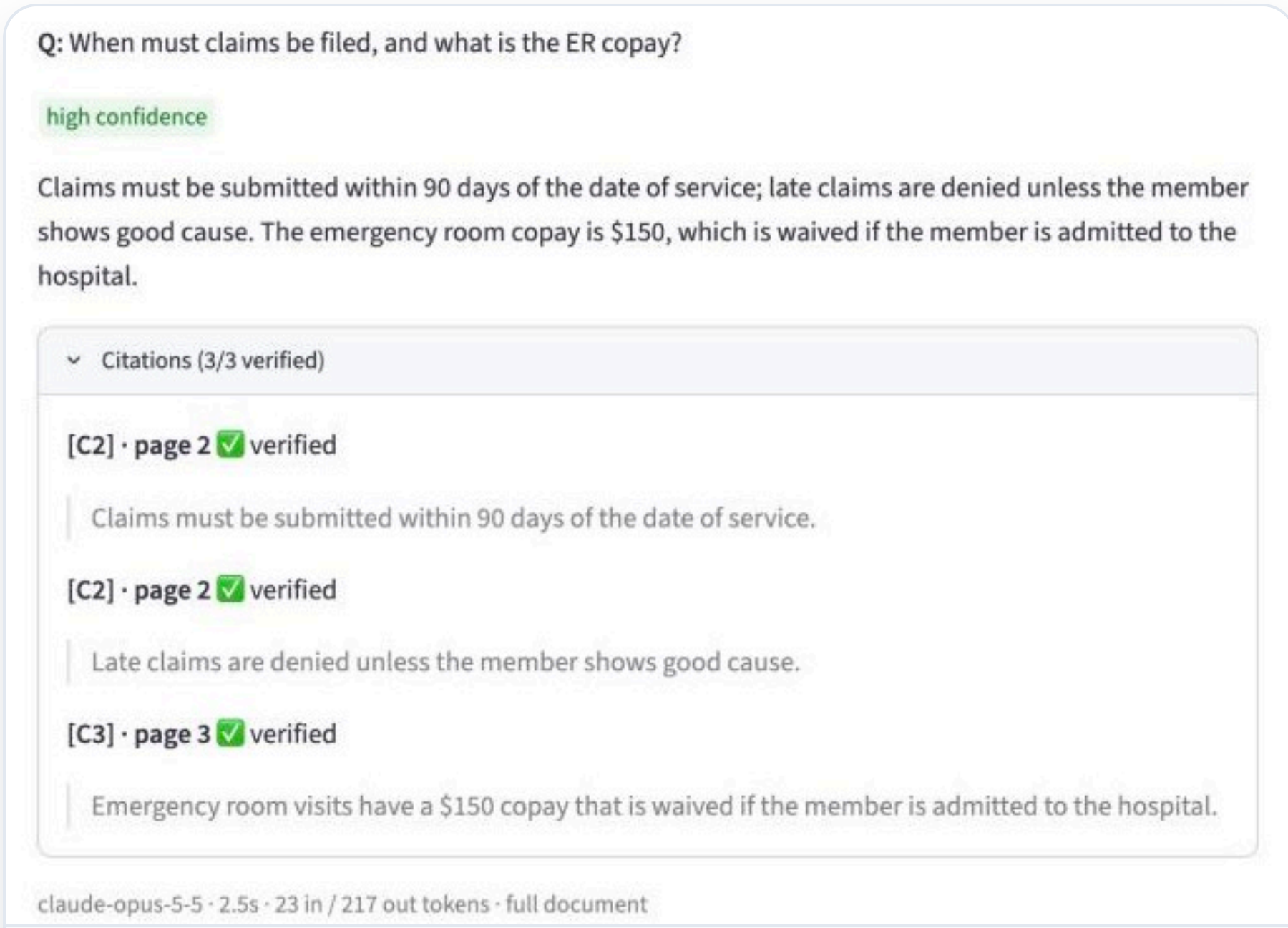
An AI can make up a source that looks just as convincing as a real one.

Unverifiable means unusable

In regulated work, an answer without a source can't be acted on.

WHAT I BUILT

Cited, quoted, checked.



Real answer from the live app, on a sample insurance policy PDF

Page number
Where each claim came from

Exact quote
The words it's based on

✅ Verified
Checked against the source in code

HOW IT WORKS

Four steps. The last one is the point.

1

Upload

A PDF or text file is split into numbered passages that keep their page numbers.

2

Ask Claude

The document and the question go to Claude Opus 5.5.

3

Structured answer

The reply must match a fixed format: answer, confidence, and a passage + quote for each claim.

4

Verify in code

Each quote is matched against the passage it cites. Anything made up gets flagged ⚠️.

RESULTS ON THE LIVE APP

Tested, not just demoed.

100%
of citations verified against the source (31 / 31)

18 / 18
questions correct, incl. 4 trick ones declined with nothing invented

3.5 s
median time to a full, cited answer

$0.005
average cost per question; caching cut it 55%

$1/day
hard spending cap on the public demo, so a shared link can't run up the bill

18-question test on three short sample documents; longer documents cost more. Python · FastAPI · Claude · Streamlit · Modal · 34 automated tests.

WHAT I LEARNED

Three lessons from shipping it.

1

Ask for structure, not prose.

Typed, structured output makes an AI far easier to build on reliably.

2

Check the AI's work in code.

Verifying every quote beats asking the model to "be careful".

3

Cap spending from day one.

A public demo needs rate limits and a hard budget. Mine stops at $1/day.

Try it yourself

Live demo and code in the comments.
Scan the code to open the app.
linkedin.com/in/moizalizai